AI Assisted Coding

Lab Assignment 13.1

Name : V Sai Krishnachaithanya
Hall Ticket no : 2303A52154

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
  - o Prompt AI to identify duplicate logic and replace it with functions or classes.
  - o Ensure the refactored code maintains the same output.
  - o Add docstrings to all functions.
- Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:
  - o Refactored code with a reusable function and no duplication.
  - o Well documented code

Prompt :

Refactored code with a reusable function and no duplication

```python
"""Refactored code with a reusable function and no duplication"""
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

```python
def calculate_rectangle_properties(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    print(f"Area of Rectangle: {area}")
    print(f"Perimeter of Rectangle: {perimeter}")

calculate_rectangle_properties(5, 10)
calculate_rectangle_properties(7, 12)
```

Python

Output :

```
✓ 0.0s

Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:
  - Identify repeated or related logic and extract it into reusable functions.
  - Ensure the refactored code is modular, easy to read, and documented with docstrings.
- Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

- Expected Output:
  - Code with a function calculate_total(price) that can be reused for multiple price inputs. Well documented code

Prompt :

Refactor the Code with a function calculate_total(price) that can be reused for multiple price inputs.

```python
""Refactor the code with a function calculate_total(price) that can be reused for multiple price inputs."""
rice = 250
ax = price * 0.18
otal = price + tax
rint("Total Price:", total)

rice = 500
ax = price * 0.18
otal = price + tax
rint("Total Price:", total)
```

```python
def calculate_total(price):
    tax = price * 0.18
    total = price + tax
    print("Total Price:", total)
```

Python

Output :

```
✓ 0.0s

Total Price: 295.0
Total Price: 590.0
```

Task Description #3: Refactoring Using Classes and Methods

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
   - Accept marks as a parameter.
   - Return the corresponding grade as a string.
   - The grading logic must strictly follow the conditions below:
     - Marks ≥ 90 and ≤ 100 → "Grade A"
     - Marks ≥ 80 → "Grade B"
     - Marks ≥ 70 → "Grade C"
     - Marks ≥ 40 → "Grade D"
     - Marks ≥ 0 → "Fail"

   Note: Assume marks are within the valid range of 0 to 100.
4. Include proper docstrings for:
   - The class
   - The method (with parameter and return descriptions)
5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.

Prompt :

Refactor the code and define a class Gradecalculator with a method calculate_grade(marks) that can be reused for different marks inputs and returns the grade instead of printing it directly.

```
"""Refactor the code and define a class Gradecalculator with a method calculate_grade(marks) that can be reused for

class GradeCalculator:
    def calculate_grade(self, marks):
        if marks >= 90:
            return "Grade A"
        elif marks >= 75:
            return "Grade B"
        else:
            return "Grade C"
grade_calculator = GradeCalculator()
print(grade_calculator.calculate_grade(95))
print(grade_calculator.calculate_grade(80))
print(grade_calculator.calculate_grade(72))

# if marks >= 90:
```

Output :

```
✓ 0.0s

Grade A
Grade B
Grade C
```

Task Description #4 (Refactoring – Converting Procedural Code)
Instructions:
o Identify input, processing, and output sections.
o Convert each into a separate function.
o Improve code readability without changing behavior.
• Sample Legacy Code:
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
• Expected Output:
o Modular code using functions like get_input(), calculate_square(), and display_result().

Prompt :

Refactor code and add functions like get_input(), calculate_square(), and display_result()

```
"""Refactor code and add functions like get_input(), calculate_square(), and display_result()"""

# num = int(input("Enter number: "))
# square = num * num
# print("Square:", square)

def get_input():
    return int(input("Enter a number: "))

def calculate_square(number):
    return number * number

def display_result(square):
    print("Square:", square)

number = get_input()
square = calculate_square(number)
display_result(square)
```

Output :

```
✓ 1.8s

Square of 123 is 15129
```

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Prompt :

Refactor the code into a class like EmployeeSalaryCalculator with methods and attributes

```python
"""refactor the code into a class like EmployeeSalaryCalculator with methods and attributes"""
class EmployeeSalaryCalculator:
    def __init__(self, salary, tax_rate=0.2):
        self.salary = salary
        self.tax_rate = tax_rate
        self.tax = None
        self.net = None

    def calculate_tax(self):
        self.tax = self.salary * self.tax_rate
        return self.tax

    def calculate_net(self):
        if self.tax is None:
            self.calculate_tax()
        self.net = self.salary - self.tax
        return self.net

    def display_net(self):
        print(self.calculate_net())

# example usage
calculator = EmployeeSalaryCalculator(50000)
calculator.display_net()
```

Output :

```
✓ 0.0s

40000.0
```

Task 6 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

o Use of sets or dictionaries with complexity justification

Prompt :

Refactor inefficient linear searches using appropriate data structures like sets or dictionaries for faster lookups.

```
"""Refactor inefficient linear searches using appropriate data structures like sets or dictionaries for faster lookups."""

users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")
found = name in users
for u in users:
    if u == name:
        found = True
print("Access Granted" if found else "Access Denied")
```

Output :

```
✓ 3.6s

Access Granted
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:
    - add_book(title, author, isbn)

    - remove_book(isbn)
    - search_book(isbn)
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
```

```
        del library_db[isbn]
        print("Book removed successfully.")
    else:
        print("Book not found.")

    # Searching again
    isbn = "101"
    if isbn in library_db:
        print("Book Found:", library_db[isbn])
    else:
        print("Book not found.")
```

Prompt :

refactor the code into a proper format with functions like add_book(), search_book(), and remove_book() to manage the library database effectively.

```python
"""refactor the code into a proper format with functions like add_book(), search_book(), and remove_book() to manage the library database effectively."""
library_db = {}

def add_book(title, author, isbn):
    if isbn not in library_db:
        library_db[isbn] = {"title": title, "author": author}
        print("Book added successfully.")
    else:
        print("Book already exists.")

def search_book(isbn):
    if isbn in library_db:
        print("Book Found:", library_db[isbn])
    else:
        print("Book not found.")

def remove_book(isbn):
    if isbn in library_db:
        del library_db[isbn]
        print("Book removed successfully.")
    else:
        print("Book not found.")

# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"

add_book(title, author, isbn)

# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"

add_book(title, author, isbn)
```

Output :

```
✓ 0.0s

Book added successfully.
Book added successfully.
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
Book not found.
Book already exists.
Book already exists.
Book not found.
Book not found.
Book not found.
```

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

Global variables.

- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
 c=a+b
 print(c)
 a=b
b=c
```

Prompt :

Refactor into a clean reusable function (generate_fibonacci) add test case

```
"""Refactor into a clean reusable function (generate_fibonacci ) add test cases"""

def generate_fibonacci(n):
    a, b = 0, 1
    print(a)
    print(b)
    for i in range(2,n):
        c = a + b
        print(c)
        a, b = b, c
n = int(input("Enter limit: "))
generate_fibonacci(n)
# Test case 1: n = 0
print("Test case 1: n = 0")
generate_fibonacci(0)

# Test case 2: n = 1
print("Test case 2: n = 1")
generate_fibonacci(1)

# Test case 3: n = 2
print("Test case 3: n = 2")
generate_fibonacci(2)

# Test case 4: n = 5
print("Test case 4: n = 5")
generate_fibonacci(5)
```

Output :

```
✓ 2.8s
0
1
1
2
3
Test case 1: n = 0
0
1
Test case 2: n = 1
0
1
Test case 3: n = 2
0
1
Test case 4: n = 5
0
1
1
2
3
```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into is_prime(n) and is_twin_prime(p1, p2).
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
 if a%i==0:
  fa=1
fb=0
for i in range(2,b):
 if b%i==0:
```

fb=1
if fa==0 and fb==0 and abs(a-b)==2:
print("Twin Primes")
else:
print("Not Twin Primes")

Prompt :

Refactor the code into is_prime(n) and is_twin_prime(p1, p2) and generate list of twin primes

```
"""Refactor the code into is_prime(n) and is_twin_prime(p1, p2) and generate list of twin primes"""
def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
def is_twin_prime(p1, p2):
    return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2
twin_primes = []
for num in range(2, 100):
    if is_prime(num) and is_twin_prime(num, num + 2):
        twin_primes.append((num, num + 2))
print("Twin Primes:", twin_primes)
```

Output :

```
✓ 0.0s
Twin Primes: [(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73)]
```

Task 10 – Refactoring the Chinese Zodiac Program
Objective
Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.
The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.
Your task is to refactor the code to improve readability, maintainability, and structure.
Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox

3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

 # Chinese Zodiac Program (Unstructured Version)
 # This code needs refactoring.

```
year = int(input("Enter a year: "))

if year % 12 == 0:
    print("Monkey")
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
    print("Goat")
```

You must:

1. Create a reusable function: get_zodiac(year)
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.

4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

Prompt :

Refactor the code to determine the Chinese Zodiac sign based on the year input using function get_zodiac(year) and return the zodiac sign instead of printing it directly.

```python
"""Refactor the code to determine the Chinese Zodiac sign based on the year input using function get_zodiac(year) and return the zodiac sign instead of printing it directly."""
def get_zodiac(year):
    zodiac_signs = [
        "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
        "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
    ]
    return zodiac_signs[year % 12]
year = int(input("Enter year: "))
zodiac_sign = get_zodiac(year)
print(f"The Chinese Zodiac sign for the year {year} is: {zodiac_sign}")
```

Output :

```
✓ 2.3s
The Chinese Zodiac sign for the year 2005 is: Rooster
```

Task 11 – Refactoring the Harshad (Niven) Number Checker
Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.
A Harshad (Niven) number is a number that is divisible by the sum of its digits.
For example:

- **18 → 1 + 8 = 9 → 18 ÷ 9 = 2 ✅ (Harshad Number)**
- **19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer ❌ (Not Harshad)**

Problem Statement
The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.
Harshad Number Checker (Unstructured Version)

```
num = int(input("Enter a number: "))
temp = num
sum_digits = 0
while temp > 0:
    digit = temp % 10
    sum_digits = sum_digits + digit
    temp = temp // 10
if sum_digits != 0:
    if num % sum_digits == 0:
        print("True")
    else:
        print("False")
else:
    print("False")
```

You must:

1. Create a reusable function: is_harshad(number)
2. The function must:
    - Accept an integer parameter.
    - Return True if the number is divisible by the sum of its digits.
    - Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

Prompt :

Refactor the code to check if a number is a Harshad number using a function is_harshad(number)accepts only integers that returns True or False instead of printing the result directly.

```
""" Refactor the code to check if a number is a Harshad number using a function is_harshad(number)accepts  only integers that returns True or False instead o

def is_harshad(number):
    temp = number
    sum_of_digits = 0
    while temp > 0:
        digit = temp % 10
        sum_of_digits += digit
        temp //= 10
    return number % sum_of_digits == 0
num = int(input("Enter a number: "))
if is_harshad(num):
    print(f"{num} is a Harshad number.")
else:
    print(f"{num} is not a Harshad number.")
```

Output :

```
✓ 1.6s
123 is not a Harshad number.
```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in n! (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

```
# Factorial Trailing Zeros (Unstructured Version)
n = int(input("Enter a number: "))
fact = 1
i = 1
while i <= n:
    fact = fact * i
    i = i + 1
count = 0
while fact % 10 == 0:
    count = count + 1
    fact = fact // 10
print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: count_trailing_zeros(n)
2. The function must:
    - Accept a non-negative integer n.
    - Return the number of trailing zeros in **n!**.

3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

Prompt :

```python
"""Refactor the given poorly structured Python script into a clean and reusable function that calculates the factorial of a number and
def factorial(n):
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    return fact
def count_trailing_zeros(n):
    count = 0
    while n % 10 == 0:
        count += 1
        n //= 10
    return count
n = int(input("Enter a number: "))
fact = factorial(n)
print(f"Factorial of {n} is {fact}")
trailing_zeros = count_trailing_zeros(fact)
print(f"Number of trailing zeros in {n}! is {trailing_zeros}")
```

Output :

```
✓ 2.8s

Factorial of 23 is 25852016738884976640000
Number of trailing zeros in 23! is 4
```

Task 13 (Collatz Sequence Generator – Test Case Design)

• Function: Generate Collatz sequence until reaching
• Test Cases to Design:
- Normal: 6 → [6,3,10,5,16,8,4,2,1]
- Edge: 1 → [1]
- Negative: -5
- Large: 27 (well-known long sequence)
  • Requirement: Validate correctness with pytest.

Explanation:
We need to write a function that:
- Takes an integer n as input.
- Generates the Collatz sequence (also called the 3n+1 sequence).
- The rules are:
  - If n is even → next = n / 2.
  - If n is odd → next = 3n + 1.

- Repeat until we reach 1.
- Return the full sequence as a list.

Example
Input: 6
Steps:

- 6 (even → 6/2 = 3)
- 3 (odd → 3*3+1 = 10)
- 10 (even → 10/2 = 5)
- 5 (odd → 3*5+1 = 16)
- 16 (even → 16/2 = 8)
- 8 (even → 8/2 = 4)
- 4 (even → 4/2 = 2)
- 2 (even → 2/2 = 1)

Output:
[6, 3, 10, 5, 16, 8, 4, 2, 1]

Prompt :

generate a python code for Collatz Sequence Generator with test cases

```python
""" generate a python code for  Collatz Sequence Generator  with test cases"""
def collatz_sequence(n):
    sequence = []
    while n != 1:
        sequence.append(n)
        if n % 2 == 0:
            n //= 2
        else:
            n = 3 * n + 1
    sequence.append(1)
    return sequence
# Test cases
print("Test case 1: n = 6")
print(collatz_sequence(6))
print("Test case 2: n = 19")
print(collatz_sequence(19))
print("Test case 3: n = 1")
print(collatz_sequence(1))
```

Output :

```
✓ 0.0s

Test case 1: n = 6
[6, 3, 10, 5, 16, 8, 4, 2, 1]
Test case 2: n = 19
[19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1]
Test case 3: n = 1
[1]
```

Task 14 (Lucas Number Sequence – Test Case Design)

• Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then Fn = Fn-1 + Fn-2)

• Test Cases to Design:

- Normal: 5 → [2, 1, 3, 4, 7]
- Edge: 1 → [2]
- Negative: -5 → Error
- Large: 10 (last element = 76).

• Requirement: Validate correctness with pytest.

Prompt :

Generate a python code for Lucas sequence up to n terms and add test cases

```python
""" Generate a python code for Lucas sequence up to n terms and add test cases"""
def lucas_sequence(n):
    sequence = []
    a, b = 2, 1
    for _ in range(n):
        sequence.append(a)
        a, b = b, a + b
    return sequence
# Test cases
print("Test case 1: n = 0")
print(lucas_sequence(5))
print("Test case 2: n = 1")
print(lucas_sequence(1))
print("Test case 3: n = 10")
print(lucas_sequence(10))
```

Output :

```
✓ 0.0s

Test case 1: n = 0
[2, 1, 3, 4, 7]
Test case 2: n = 1
[2]
Test case 3: n = 10
[2, 1, 3, 4, 7, 11, 18, 29, 47, 76]
```

Task 15 (Vowel & Consonant Counter – Test Case Design)

• Function: Count vowels and consonants in string.

• Test Cases to Design:

- Normal: "hello" → (2,3)
- Edge: "" → (0,0)
- Only vowels: "aeiou" → (5,0)

Large: Long text

• Requirement: Validate correctness with pytest.

Prompt :

Generate a python code for Count vowels and consonants in string. Add test cases

```python
"""Generate a python code for Count vowels and consonants in string. Add test cases"""
def count_vowels_consonants(s):
    vowels = "aeiouAEIOU"
    vowel_count = 0
    consonant_count = 0
    for char in s:
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
    return vowel_count, consonant_count

# Test cases
print("Test case 1: 'Hello World'")
v, c = count_vowels_consonants("Hello World")
print(f"Vowels: {v}, Consonants: {c}")

print("Test case 2: 'Programming'")
v, c = count_vowels_consonants("Programming")
print(f"Vowels: {v}, Consonants: {c}")

print("Test case 3: 'AEIOU'")
v, c = count_vowels_consonants("AEIOU")
print(f"Vowels: {v}, Consonants: {c}")
```

Output :

```
✓ 0.0s

Test case 1: 'Hello World'
Vowels: 3, Consonants: 7
Test case 2: 'Programming'
Vowels: 3, Consonants: 8
Test case 3: 'AEIOU'
Vowels: 5, Consonants: 0
```